



Beyond RAG: Advanced Agentic Knowledge Architectures

Week 2: From Search to Intelligence

Summary

SupportVectors AI Lab

Copyright Notice

© SupportVectors. All rights reserved.

This document is the intellectual property of SupportVectors, and part of its training material. Only the participants in SupportVectors workshops are allowed to study the document for educational purposes currently, but is prohibited from copying or using it for any other purposes without written permission.

Contents

1	The Paradigm Shift in Information Retrieval	4
1.1	The Transmission of Human Knowledge	4
1.2	From Library to Librarian	4
1.3	The Hallucination Paradox and Statistical Priors	5
1.4	Retrieval-Augmented Generation as a Bayesian Machine	5
2	Information Geometry and the Native Tongue of Machines	5
2.1	The Enduring Nature of Mathematics	5
2.2	Mapping Thought to a Hyper-Globe	6
2.3	The Necessity of High Dimensions	6
2.4	Engineering Constraints and Lossy Compression	6
2.5	The Curse of Dimensionality and Concentration of Measure	7
3	The Mechanics of Machine Learning: Probability, Surprise, and Gradients	7
3.1	The Masked Language Modeling Game	7
3.2	Navigating Linguistic Context	9
3.3	The Bonus Round: Unidirectional vs. Bidirectional Vision	10
3.4	Loss Functions and Aleatoric Uncertainty	10
3.5	Optimization Through Gradient Descent	11
4	Deep Bidirectional Transformers and Text Embeddings	11
4.1	The BERT Architecture	11
4.2	Pre-training vs. Fine-Tuning (Transfer Learning)	12
4.3	Pooling: Distilling Context into Text Embeddings	13
5	Continuous Learning and the Future of Knowledge Architectures	14
5.1	Derivative Artifacts	14
5.2	Fluid Embeddings and Self-Evolving Agents	14
6	The Foundations of Probability, Surprise, and Negative Log Likelihood	15
6.1	The Language Game and Probability Sand	15
6.2	The Mathematical Definition of Surprise	15
6.3	Negative Log Likelihood (NLL)	16
7	Logits, Softmax, and Temperature	17
7.1	Logits and The Stadium Analogy	17
7.2	The Three-Step Softmax Process	17
7.3	The Coding Exercise and the Effect of Temperature	18
7.4	Sampling the Softmax and Determinism	19
7.4.1	The Temperature Paradox in RAG	19
7.4.2	Lexical Variance vs. Semantic Consistency	20

7.4.3	Hardware Indeterminacy	20
7.5	Theory of the Mind in LLMs	21
8	The Attention Mechanism	21
8.1	The Dictionary Lookup Analogy	21
8.1.1	The Spectrum of Meaning: Where Does It Live?	22
8.2	The Attention Heat Map	23
8.3	Machines Speak Machineese: Queries, Keys, and Values	24
8.4	The Core Equation of Attention	25
8.5	Self-Attention and “Attention is All You Need”	26
8.5.1	Self-Attention: The Small Dictionary	26
9	Capstone Projects and The Embedding Histograms Lab	27
9.1	The Baseline RAG Capstone	27
9.2	The Embeddings Evaluation Lab	27
10	Key Takeaways	29

“
You shall know a word by the company it keeps.
”

John Rupert Firth

(1890–1960), English linguist whose theory of context-dependent meaning laid the conceptual foundation for modern word embeddings.

1 The Paradigm Shift in Information Retrieval

1.1 The Transmission of Human Knowledge

The survival and advancement of the human species rely entirely on the transmission of knowledge from one generation to the next. Unlike lions or tigers, humans do not possess biological defenses such as fangs or claws; humanity inherits knowledge. As the volume of knowledge grows, the relevant segments of that knowledge must remain accessible. Whether stored as drawings on a cave wall, within the scrolls of the Library of Alexandria, among the archives of the Library of Congress, or across the great digital library of the World Wide Web, knowledge is practically non-existent if it cannot be retrieved when needed. Therefore, search architectures hold a uniquely profound place in the development of civilization.

1.2 From Library to Librarian

Before 2023, the search industry operated fundamentally like a vast library index. Whether searching the open internet or an internal corporate intranet via systems like Elastic Search or Solr, the modality remained the same. A query composed of keywords was entered, and the system returned a list of URLs. The responsibility of drilling down into those URLs, reading the material, understanding it, and manually reconstructing an answer fell entirely on the user.

With the mass adoption of Large Language Models (LLMs), a fundamental shift occurred. Language models possess the ability to understand human language, raising a critical question: why shouldn't these models read the underlying documents, parse the web pages or videos, and figure out the answers directly? Fusing retrieval architectures

with language models gave a voice to previously dormant, silent corpuses of data, making them eloquent. The transition was akin to moving from a traditional library where one must physically pull books from the stacks, to a library staffed by an omniscient librarian who instantly synthesizes and delivers the exact answer required.

1.3 The Hallucination Paradox and Statistical Priors

Despite their fluency, LLMs face a fundamental limitation technically known as hallucination. When a language model is asked a direct question, for example, “What did I have for breakfast today?”, it utilizes a statistical prior. By ingesting all of human literature and accumulated knowledge, the model recognizes patterns. It might deduce that individuals in a specific demographic frequently consume coffee and bagels. Based on the observation of a coffee pot in a breakroom, guessing “coffee” is absolutely logical.

The resulting answer is highly plausible, confidently delivered, and yet entirely incorrect for that specific day, except by sheer accident. When a user queries a language model, the safe presumption is that the user does not know the answer. The paradox and danger arise when the model confidently presents a statistically plausible “prior belief” as an absolute truth.

1.4 Retrieval-Augmented Generation as a Bayesian Machine

Retrieval-Augmented Generation (RAG) fundamentally solves the hallucination paradox by operating as a Bayesian machine. In a Bayesian framework, a system moves from a prior belief to a highly accurate posterior belief through the introduction of specific evidence.

While the language model brings its prior beliefs encoded within its parametric memory (its weights and biases), a search engine intervenes by providing a crucial piece of evidence, a retrieved document explicitly stating, for instance, that the specific breakfast consisted of Indian bread, coffee, and pickles. By incorporating this precise evidence, the language model is steered away from the generalized answer it would have given otherwise. It incorporates the new data to form a highly accurate posterior belief, delivering a factual, contextually grounded response.

2 Information Geometry and the Native Tongue of Machines

2.1 The Enduring Nature of Mathematics

Engineering tools, software libraries, and specific vector databases are subject to the practicalities and exigencies of their time. Software notoriously goes through iterations: version one is when it might work, version two is when it begins to work, and version three is when it actually functions. However, there is no “version two” of the Pythagorean

theorem. Mathematics is applied reasoning and logic; once a theorem is established, it is intrinsically true forever. Therefore, understanding knowledge architectures requires looking beyond temporary coding frameworks and focusing on the enduring mathematical foundations of information geometry.

2.2 Mapping Thought to a Hyper-Globe

Text written in any human language, English, Spanish, Mandarin, Tamil, Hindi, or French, must be converted into “Machineese,” the native tongue of artificial intelligence. In Machineese, every thought or piece of text is translated into a coordinate point on a high-dimensional hyper-globe.

When a text is processed, the language model translates it into a geographical location on this mathematical structure. For instance, a document discussing physics might map to a region equivalent to “Australia” on this globe, while a biological text maps to “Europe,” and another concept lands in “Uganda.” This spatial placement dictates semantic embedding: the geometrical location speaks directly to the semantic content of the text. Models are explicitly designed to ensure that similar thoughts are kept strictly in nearby neighborhoods.

2.3 The Necessity of High Dimensions

To differentiate simple concepts, low dimensions suffice. To separate a cow from a duck, a two-dimensional grid plotting longitude for weight against latitude for volume works perfectly. Starting from a zero-weight, zero-volume South Pole, cows and ducks will naturally map to entirely different locations.

However, a cow and a sofa are both heavy and voluminous. To prevent the machine from confusing a piece of furniture with a bovine, a critical distinction, as sitting on a cow is an unpleasant experience, whereas water buffalo are friendly enough for a child to sleep on, additional dimensions are required to capture nuanced traits.

Human words are massive compressions of data. The single word “cow” carries an immense semantic payload: its physical size, eating habits, gentle eyes, swishing tail, milk production, and calves. Certain cultural texts, such as Indian *Mahavakyas* (great sayings), mantras, or ayats, contain such dense semantic compression that unpacking their full meaning requires a lifetime of study. To capture all the texture and nuances of these compressions, extremely high-dimensional spaces are mandatory.

2.4 Engineering Constraints and Lossy Compression

Despite the need for high dimensionality, machines cannot utilize infinite dimensions. Representing the simple sentence “The cow jumped over the moon” with a 1-million-dimensional vector would require 4 million bytes (at 32-bit floating-point precision) for

a single thought. This creates a compute and storage catastrophe, as calculating the distance (e.g., cosine dissimilarity or Euclidean distance) between points would become overwhelmingly expensive.

Consequently, translating human language to embeddings is a “lossy” semantic compression. It is not as perfectly lossless as an optical fiber transmitting error-corrected bits and bytes, but it avoids the cascading errors found in the children’s game of “Telephone.” Models compromise by representing text in a reasonable number of dimensions that capture *most* of the meaning.

2.5 The Curse of Dimensionality and Concentration of Measure

High-dimensional spaces also suffer from the “Curse of Dimensionality.” In vast dimensional spaces, everything is drawn incredibly far apart. The concept of a neighborhood becomes extremely weak. It is akin to living in the rural countryside of Wyoming, where speaking to a neighbor requires a one-hour drive in a truck, compared to the Bay Area, where one can simply knock on a shared wall. Randomly distributed points will inherently be distant from one another.

However, this high dimensionality offers a mathematical blessing known as the Concentration of Measure. If a specific data point acts as a “polar bear” sitting at the North Pole, random vectors are naturally repelled away from the pole, congregating in an equatorial band. Therefore, if the polar bear manages to find *any* neighboring points near the Arctic Circle, it is a certainty that those points are not there by random chance. They are deeply related semantically.

This purity of signal is the singular breakthrough that revolutionized the search industry between 2018 and 2020. Semantic search is fundamentally AI search: it operates on the profound geometric principle that proximity in high-dimensional space equates directly to meaningful answers.

3 The Mechanics of Machine Learning: Probability, Surprise, and Gradients

3.1 The Masked Language Modeling Game

The mechanism by which machines understand the nuances of language and determine where to place points on the globe can be illustrated through a masked language modeling exercise.

The core mechanics of this predictive challenge are structured as a card game titled “Guess the Missing Word: A card game in three languages.”

The methodology for playing is precise:

- One sentence is presented per card, with exactly one word missing. The missing word is always one of the 10 vocabulary words available in that deck.
- The player is given 10 chips (or dimes) per card. These chips must be distributed over the 10 words based entirely on the player's internal belief of what the correct answer is. The probability p assigned to any word is the number of chips divided by 10.
- At the reveal, the performance is evaluated using a metric called “surprise,” calculated as $-\log_2(p)$ based *only* on the chips placed on the correct word.
- The lowest total surprise wins across three sequential decks: Tibetan, Spanish, and English.
- Crucially, if zero chips are placed on the absolute truth, the calculation becomes $-\log_2(0)$, resulting in infinite surprise (∞). In this scenario, the player loses on the spot.

How to play

- One sentence per card, one word missing — always one of that deck's 10 vocabulary words.
- You get 10 chips per card. Spread them over the 10 words by belief: chips $\div$ 10 = your probability.
- At the reveal, your surprise = $-\log_2(p)$ for the chips you placed on the correct word.
- Lowest total surprise wins. Decks in order: Tibetan $\rightarrow$ Spanish $\rightarrow$ English.
- **Zero chips on the truth = $-\log_2(0)$ = infinite surprise. You lose on the spot.**

Figure 1: Rules for the game: “Guess the Missing Word: A card game in three languages.”

The mathematical distribution of this game cannot be manipulated. The chip split that minimizes the expected surprise represents the true belief. The surprise table outlines the exact penalty for uncertainty:

- Placing all 10 chips on the correct word yields 0.00 bits of surprise.
- A completely random guess across all 10 words (1 chip each) establishes a “clueless baseline” of 3.32 bits.
- Progressive confidence (e.g., 5 chips yielding 1.00 bit, 8 chips yielding 0.32 bits) mathematically rewards accuracy.

The surprise table · $-\log_2(p)$											
Chips	0	1	2	3	4	5	6	7	8	9	10
p	0	.1	.2	.3	.4	.5	.6	.7	.8	.9	1
Surprise (bits)	∞	3.32	2.32	1.74	1.32	1.00	0.74	0.51	0.32	0.15	0.00

Know nothing? One chip on each word — 3.32 bits, guaranteed. That is the clueless baseline.

Honesty pays: the chip split that minimizes your expected surprise is your true belief. This game cannot be gamed.

Figure 2: Surprise Table

3.2 Navigating Linguistic Context

When encountering a completely foreign language, the lack of context dictates performance. For instance, Card T1 from the Tibetan deck presents a sentence in Tibetan script with a blank (“nga bod pa _____”), alongside 10 transliterated vocabulary options such as *nga*, *khong*, *bod pa*, *ja*, and *red*.



Figure 3: Card T1 from the Tibetan Deck.

Because the text holds no semantic meaning to a non-speaker, any guess results in infinite, or near-infinite, total surprise. The entity cannot parse the subtext. Conversely, Card E5 from the English deck presents a highly contextualized sentence: “He is the most _____ scientist in the world.” The 10 options include *teacher*, *student*, *university*,

library, book, tea, person, interesting, delicious, and famous.

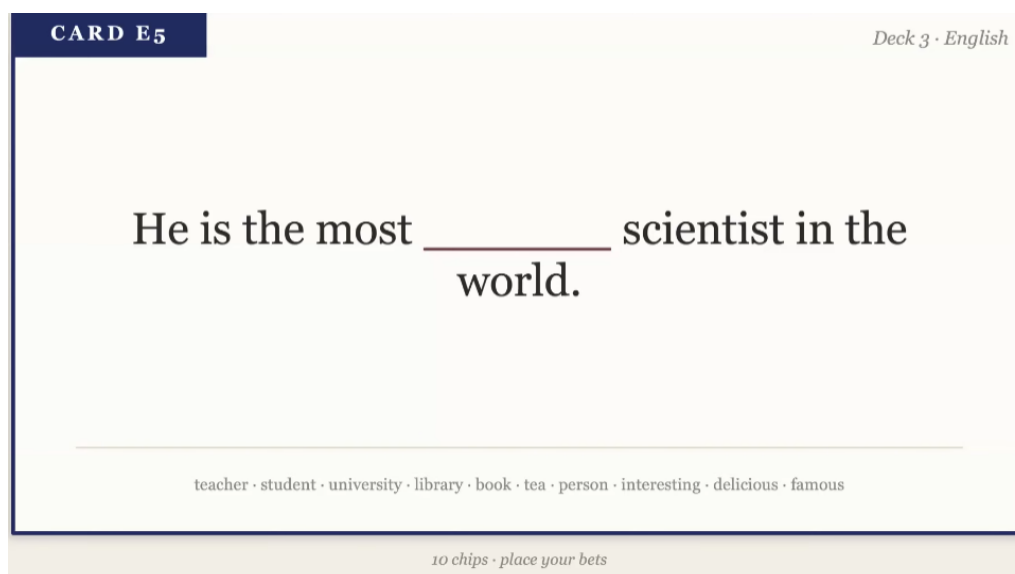


Figure 4: Card E5 from the English deck.

Because the context is overwhelmingly clear to an English speaker, the maximum number of chips is placed on “famous,” resulting in a total surprise of zero.

3.3 The Bonus Round: Unidirectional vs. Bidirectional Vision

An important architectural distinction becomes apparent during advanced rounds of this exercise. If a sentence is truncated and a predictive system is only permitted to see the text to the left of a blank, its logic is constrained. For instance, given only the words “I poured...”, the most statistically plausible guesses might be “tea” or “coffee.”

However, if the system is allowed to see the full text, both to the left and to the right of the blank, the correct answer may completely change to “book.” **A machine that possesses bidirectional vision, capable of viewing the complete surrounding context, is fundamentally more powerful at resolving ambiguity than a machine restricted to unidirectional, left-to-right vision.**

3.4 Loss Functions and Aleatoric Uncertainty

In machine learning, this aggregate “Surprise” metric is formally known as a **Loss Function**. It is a singular, powerful number indicating how poorly a system understands a language. Imagine an entity, whether an algorithm or a human in a Chinese room, hidden behind a curtain, repeatedly attempting to predict missing words. If the entity continually generates infinite net surprise, it possesses no understanding. If the surprise progressively decreases over successive attempts, transitioning from high numbers down to near zero, it is empirical evidence that the entity is learning statistical patterns and comprehending the language.

Sometimes, surprise cannot reach absolute zero due to **Aleatoric Uncertainty**, an incurable, inherent unpredictability in a system. For instance, predicting whether a person picked up a spoon or a fork at a dinner table contains fundamental uncertainty that no amount of contextual learning can perfectly resolve, as both are equally likely. Similarly, in an enterprise RAG system, while the broad domain (e.g., semiconductors) is known, the exact query a user will input in production remains inherently unknowable.

3.5 Optimization Through Gradient Descent

To actively learn, a machine must evaluate its predictions ($\hat{Y}$, pronounced “Y hat”) against the correct answers, known as the ground truth or target (Y). The machine focuses on its most egregious mistakes, the guesses that generated the highest surprise, using them as a guide.

To improve, the machine computes the **Gradient**, which is the sensitivity of the loss to the machine’s internal dials (parameters). By understanding exactly how turning a specific dial left or right impacts the overall loss, the machine systematically adjusts its parameters to drive the error downward. This optimization process is known as gradient descent, the core technique that enables machine learning models to transition from profound ignorance to high-level language comprehension.

4 Deep Bidirectional Transformers and Text Embeddings

4.1 The BERT Architecture

The classroom guessing game directly mirrors the fundamental mechanics of Masked Language Modeling (MLM), the architecture that powers landmark AI models. This concept was formalized in a seminal research paper titled “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” authored by researchers at Google AI Language [Devlin et al., 2018].

The abstract of the paper defines BERT (Bidirectional Encoder Representations from Transformers) as a novel language representation model designed to pre-train deep representations from unlabeled text by jointly conditioning on both the left and right context in all layers.

This bidirectional capability is paramount. Older causal transformers could only look to the past (the left of a blank space) to predict the missing word. However, a machine’s predictive power increases exponentially when it has bidirectional vision, allowing it to look at the text both preceding and following a mask to derive precise meaning. BERT proved empirically powerful, pushing the state-of-the-art results on eleven natural language processing benchmarks, including GLUE and MultiNLI.

810.04805v2 [cs.CL] 24 May 2019

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **Bidirectional Encoder Representations from Transformers**. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers

Figure 5: Illustration explaining the bert architecture.

4.2 Pre-training vs. Fine-Tuning (Transfer Learning)

The AI industry relies on the principle of Transfer Learning, driven by neural scaling laws. Generally, the larger the model and the more data it is fed, the more capable it becomes. However, training massive models requires compute budgets in the millions, resources restricted to hyperscalers and massive tech corporations.

The solution is split into two phases:

1. **Pre-training:** Hyperscalers utilize surrogate tasks, such as Masked Language Modeling. Taking vast amounts of scraped, unlabeled web data, the system deliberately masks words and forces the massive neural network to guess them, looping through gradient descent until the loss function minimizes. While building a “fill-in-the-blank” application has no immediate business utility, the surrogate task successfully imbues the model with a core, foundational understanding of human language.
2. **Fine-tuning:** Practitioners take these pre-trained, open-weight models and adapt them to highly specific downstream tasks. Because the pre-trained model already speaks fluent “Machinese,” it requires only a few additional output layers and minimal task-specific data to become a state-of-the-art classifier (e.g., analyzing customer reviews) or a question-answering system.

4.3 Pooling: Distilling Context into Text Embeddings

When a sentence enters an attention-based transformer network, every single word actively searches for its full meaning. Due to polysemy (words having multiple meanings), a word like “bank” will mathematically analyze surrounding words like “river,” “swam,” and “across” to pull context, emerging as a fully contextualized vector.

However, an engineering challenge arises: how does a system represent an entire multi-word sentence as a single coordinate point (a Text Embedding) on the hyper-globe? This reduction process is called **Pooling**, meaning creating one out of many.

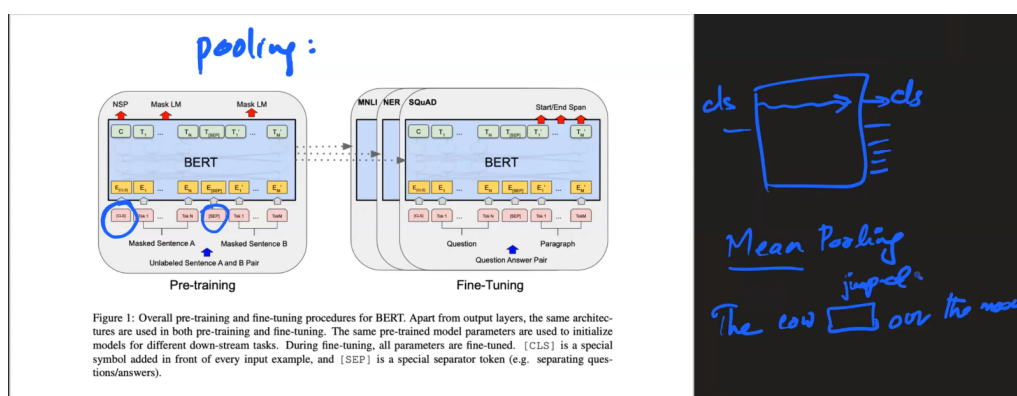


Figure 6: Pooling illustration.

The architecture diagrams outline two primary methods for pooling:

1. **The [CLS] Token (The Elected Representative):** A special, inherently meaningless classification token ([CLS]) is prepended to the beginning of the text, alongside a separator token ([SEP]). Because [CLS] holds no innate dictionary meaning, it spends the entire processing cycle aggressively soaking up the context of every other word in the sentence. When it emerges from the network, it serves as the text embedding, an “elected spokesperson” or congressman that perfectly captures the overall semantic meaning of the sentence.
2. **Mean Pooling (The Collective Average):** Alternatively, the system can mathematically aggregate all the contextualized vectors of every word in the sentence and divide by N (the total number of tokens). This calculates the precise average mathematical meaning of the entire thought. Over time, subsequent research has demonstrated that Mean Pooling generally provides a superior, more robust text embedding.

Once this pooled text embedding is generated, it serves as the input vector for any downstream machine learning algorithm, allowing systems to easily compute entailment, contradictions, or direct question-answer relevance.

tip

***The Golden Age of Decoding Research** Research papers detailing these architectures, such as the BERT publication, are often written in highly formal, dense academic language. Reading these papers linearly from abstract to conclusion is an outdated and inefficient methodology. Instead, papers should be consumed iteratively and interactively. The modern ecosystem provides tools like Google NotebookLM, Claude, and Gemini to assist in this process. By feeding a complex PDF into these systems, the dense text can be instantly converted into a conversational podcast, allowing practitioners to absorb the core ideas while exercising or hiking. Because LLMs possess the equivalent of Ph.D.-level knowledge across every specialized field, they bring immense surrounding context to the reading process. Practitioners can engage in a dialogue with the AI, asking it to explain specific paragraphs, synthesize blogs, generate infographics, or clarify complex mathematical notations, thereby democratizing access to state-of-the-art research.*

5 Continuous Learning and the Future of Knowledge Architectures

5.1 Derivative Artifacts

Standard out-of-the-box RAG implementations typically chunk and process raw documents directly. While suitable for simple proofs of concept, applying generic downloaded models to raw text generally yields poorer retrieval quality and is insufficient for world-class enterprise systems.

Advanced architectures construct **Derivative Artifacts**. In these systems, the AI is programmed to invest computational effort to deeply analyze, cross-reference, and understand the corpus *before* it ever answers a user query. This process of generating derivative artifacts lays the foundation for highly accurate, agentic RAG systems.

5.2 Fluid Embeddings and Self-Evolving Agents

Human cognition involves continuous learning; the brain solidifies connections and deepens understanding over time, even during sleep when no new external information is actively flowing in. The ultimate trajectory of RAG architectures is to mimic this behavior through self-evolving agents.

In basic systems, a text string's vector coordinates are fixed. In advanced, continuous-learning architectures, **Fluid Embeddings** are utilized. As the system processes data and evaluates connections, its internal understanding evolves. If the system initially

places concept X , Y , and Z equidistantly, but later realizes that X is fundamentally related to Z , the embedding space actively updates. The coordinate point for X will physically migrate through the vector space, moving closer to Z and further from Y . This allows the RAG architecture to achieve true continuous learning, refining its geometric understanding of knowledge autonomously over time.

6 The Foundations of Probability, Surprise, and Negative Log Likelihood

6.1 The Language Game and Probability Sand

The post-lunch session opened with a reflection on the morning's language exercise, where the class played a "fill in the blanks" game using Tibetan, Spanish, and English sentences. Two primary human behaviors emerged from this exercise:

1. **Accumulating Surprise:** When students didn't know Tibetan, they accumulated a massive amount of "surprise." Conversely, for the English sentences, the surprise was essentially zero.
2. **Spreading vs. Snapping:** When uncertain, humans instinctively "hedge their bets," spreading their guesses (their conceptual "dimes") across multiple potential words. When confident in English, humans piled all their chips onto a single correct word.

This human intuition maps perfectly to how machine learning models operate. When an LLM predicts the next word, it generates a **probability mass distribution**. Think of probability as a bucket of sand. You pour different amounts of sand into bowls representing possible words. Because you have exactly one bucket of sand, this introduces the **Law of Conservation of Probability**: The sum of all probabilities assigned across all possible outcomes must strictly equal 1.

6.2 The Mathematical Definition of Surprise

Let us introduce the emotion of "surprise" as a mathematical variable. If a student holds a prior belief that there is a 0% chance of rain, and it rains, their surprise is infinite. If they believe there is an 80% chance of rain, their surprise is very low. Thus, surprise and probability have an inverse relationship.

The class brainstormed mathematical formulas for surprise:

- $1/p$: This fails because if $p = 1$ (100% certainty), the surprise is 1, but it should intuitively be 0.
- $(1/p) - 1$: This works mathematically, but there is a deeper, more elegant solution.

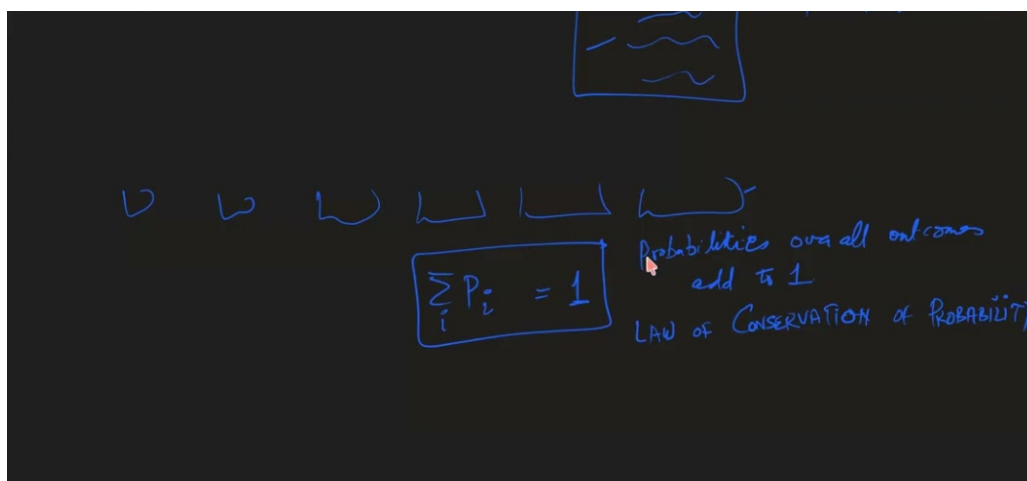


Figure 7: Law of conservation of Probability.

The industry standard is the logarithmic approach:

$$\text{Surprise} = \log(1/p) = -\log(p) \quad (1)$$

Logarithms are monotonically increasing and elegantly map a probability of 1 to a surprise of 0, while a probability approaching 0 yields a surprise approaching infinity.

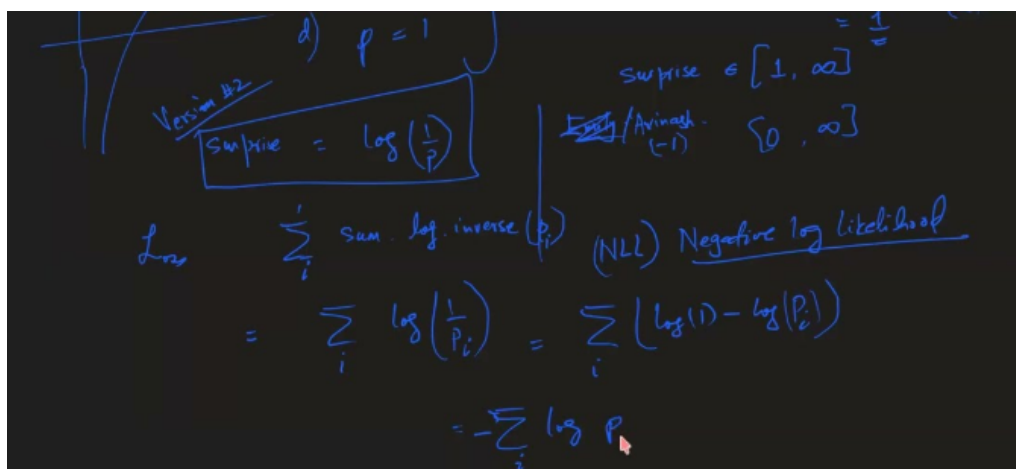


Figure 8: Blackboard explanation of Surprise and Negative Log Likelihood(NLL)

6.3 Negative Log Likelihood (NLL)

When a machine makes predictions across a batch of 32 independent sentences, its success is measured by the **joint probability** (the likelihood) of getting all answers correct. The joint probability of independent events is their product ($P_1 \times P_2 \times P_3 \dots P_{32}$).

If you ask a computer to multiply 32 small probabilities (e.g., numbers between 0 and 1), the result quickly hits **numerical underflow** (e.g., 8×10^{-15}). Computers cannot

process numbers this infinitesimally small; they round them to zero, making optimization impossible.

The logarithm acts as a magician, converting multiplication into addition:

$$\log(P_1 \times P_2 \dots) = \log(P_1) + \log(P_2) \dots \quad (2)$$

By taking the log, these numbers become manageable negative sums (e.g., -33.27). To train a model to **maximize** the log likelihood (Maximum Likelihood Estimator, MLE), we simply tell optimization algorithms to **minimize** the negative of that sum.

Summing surprise across multiple independent events (e.g., the five sentences from the morning exercise) gives the **total surprise**, which is identical to the **loss** used in training models. Using the identity $\log(1/p) = -\log(p)$, the total loss becomes:

$$\text{Loss} = - \sum_i \log(p_i) \quad (3)$$

This is the famous **Negative Log Likelihood (NLL)**. Thus, minimizing **Negative Log Likelihood (NLL)** is mathematically identical to minimizing the sum of the machine’s “surprises.” NLL serves as the foundational loss function for training language models and classifiers.

7 Logits, Softmax, and Temperature

7.1 Logits and The Stadium Analogy

Machine learning models output raw, unbounded numbers called **logits**, which can range from $-\infty$ to $+\infty$. For example, when fans watch a player, they cheer (positive numbers) or boo (negative numbers). Similarly, if a machine is classifying an image of an animal among 10 options, it might output a logit of -65 for “zebra” (strong disbelief) and $+100$ for “duck” (strong belief).

However, logits are not probabilities. To convert logits into a probability distribution, we use the **Softmax** function—referred to as the “Cheshire Cat” of machine learning because it reliably appears across all deep learning architectures.

7.2 The Three-Step Softmax Process

The Softmax operation follows a strict three-step sequence to transform logits into probabilities:

1. **Scale by Temperature (X_i/T):** Divide the logits by a hyperparameter T .
2. **Positivize ($e^{X_i/T}$):** Exponentiate the scaled logits. Exponentiation preserves the magnitude order (e.g., -2 remains larger than -3) but converts all outputs to

positive numbers. $e^{-\infty} \rightarrow 0$.

3. **Normalize:** Divide each exponentiated value by the sum of all exponentiated values (the Partition Function, Z). This ensures all values sum exactly to 1.

The complete Softmax equation is:

$$P(X_i) = \frac{e^{X_i/T}}{\sum_j e^{X_j/T}} \quad (4)$$

where $Z = \sum_j e^{X_j/T}$ is the partition function.

7.3 The Coding Exercise and the Effect of Temperature

The class performed a programmatic exercise, generating 10 random logits and applying softmax across various temperatures:

- **Low Temperature ($T = 0.01$ or 0.1):** The machine becomes overconfident. It “snaps hard,” pouring 100% of its probability sand onto the largest logit and visually wiping out all other options. This mimics a “hardmax.”
- **Medium Temperature ($T = 1$):** A standard distribution. The machine strongly favors the top logit but leaves room for the second and third.
- **High Temperature ($T = 10$ or 100):** The machine “hedges its bets.” The probabilities spread out broadly and uniformly. Crucially, the size order of the logits is strictly preserved, but the gaps between them are heavily muted.

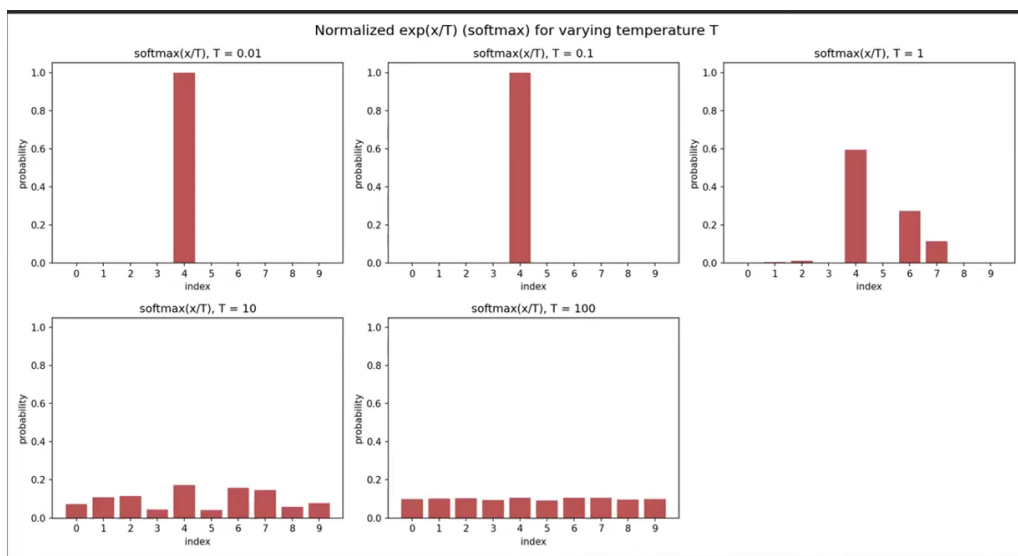


Figure 9: Softmax Subplots.

7.4 Sampling the Softmax and Determinism

A neural network is at the end of the day a mathematical function. A function by definition should produce exactly the same output for the same input over and over again. If you wrote a square root function and every time you gave it the input 9 it sometimes produced 3, sometimes 4, sometimes 2—that would not be a good function. So a neural network is inherently deterministic.

But language models don't behave that way. Every time you ask the same question, you get a different answer lexically, though the semantic core remains stable. Why would people destroy determinism? Because they noticed that without something called sampling, the model generated boring and repetitive answers. It would get into a loop and keep repeating itself. In the early days of ChatGPT 3.5, if you asked something very complex, it would sometimes start repeating sentences.

The intuition behind sampling is this: imagine there is probability sand lying in bowls, and you send a kid in a hurry to quickly pick the bowl with the most sand. The kid is likely to pick up the heaviest bowl, but sometimes—especially when the differences between bowls are small (high temperature)—the kid might grab the second or third heaviest instead. That is sampling. You're more likely to pick the highest-mass item, but the kid may pick any one of them.

Once a token is sampled, it commits the model to that path. If the question is “I have \$10, ice cream costs \$4 and pizza costs \$2, should I have ice cream?” and the first sampled token is “Yes,” the model now has to justify that answer. If instead it had sampled “No,” it would have justified the opposite. That is how variability comes into the machine.

If you don't want sampling, that is called the **greedy approach**—you turn off sampling, and it basically becomes hardmax. You can set the temperature to zero, which turns off sampling altogether, but generally that's not considered a good idea unless you are in a very strict domain like legal.

Temperature as the creativity knob: The higher the temperature, the more likely the model is to go in unexpected directions. The lower the temperature, the more it behaves like a deterministic machine.

7.4.1 The Temperature Paradox in RAG

Student's question: “If I build a RAG system, I would want it to give the same answer over and over again. Why would I leave sampling on? Why would I leave temperature at, say, 1?”

The instructor posed this as a paradox: if you ask “How many days of PTO can I take?” in an HR application, and one day the system says “I think 40 is just about right for you” and another day says “No, you're only allowed five”—that would be a disaster.

And yet, setting temperature to zero is usually not a very good idea.

The answer is nuanced. For point-answer situations (like the PTO question), yes, you can set it low. But generally, wherever reasoning, thinking, or evaluation is involved, it is not a good idea. In fact, when GPT-5 first came out, OpenAI explicitly would not let you set the temperature below 1. It was a reasoning model—its job was to tell you the answer to math problems—and yet they forced temperature to 1. If you tried temperature zero, it would bail on you. (GPT 5.2 onwards they changed it back and now you can set different temperature values.)

Why would a company do that? Because these language models are not just next-token prediction machines. They are complex machines that internally have concepts. There is a paper from Anthropic on polysemanticity showing that if you open up the layers, you can see concepts forming. There is another paper from Apple Research showing that when the model predicts a token—say it produces “cow” in “the cow jumped over the moon”—if you look at the probability sand that accumulated on the word “jumped,” it is already rising even though the model hasn’t produced it yet. The machine is already thinking through the whole sentence and producing it one by one. This whole movement is called **mechanistic interpretability**, and it remains a deep, active area of research.

7.4.2 *Lexical Variance vs. Semantic Consistency*

To demonstrate the distinction, the instructor ran a classroom experiment. He said: “Yesterday evening, after a hot day, from the Pacific, fog came in and rolled over the San Francisco hills.” Then he asked students to repeat the same thought in their own words:

- Student A said: “Fog came in and rolled over the area.”
- Student B said: “San Francisco Bay area, fog rolled over the hills.”

Two human beings expressed the same semantic content using completely different words. LLMs behave exactly the same way—they preserve very stably the semantic content, but on repeated asking of the same question, the specific word constructions change while what they mean remains the same.

Even for “2+2,” the answer is always semantically “four,” but the model might say “Four,” or “Four is a number,” or “The sum is four,” or “It adds up to four.”

7.4.3 *Hardware Indeterminacy*

It is a common misconception that if you set the temperature to zero, you get a fully deterministic machine. Actually, you don’t. These models are ginormous. They run on big GPUs. Those GPUs share their weights, and their numerical rounding issues, floating-point precision limits, and the sequence in which the answers arrive for matrix

accumulation and computation all introduce indeterminacy. You never get perfect determinism even when you turn off sampling altogether. This has to do with the nature of the hardware and its inherent indeterminacy.

Don't ever set the temperature to zero in your RAG systems and then assume your answers will always be the same. It will surprise you, and it will definitely surprise you when you are showing it to your client—Murphy's Law. The best you can do is get as close as possible within the margins of the hardware indeterminacy.

7.5 Theory of the Mind in LLMs

If you ask an LLM “What is $2+2$?” it just says “4.” But the instructor posed a different prompt: “I'm a number theorist. What does it mean to say $2+2$ is 4?” The model's answer changed drastically—it began discussing abstraction, deep philosophers of science and mathematics like Bertrand Russell and Whitehead who laid the foundation of axiomatic mathematics. It dwelt upon what it means to define “1,” to define “2,” what “equal to” means. It noted that in binary arithmetic there is no four, there is no two, and that in modular 3 arithmetic $2+2$ is not four—it is one. There are many implicit assumptions, and if you change the context, the model answers differently.

This is because machines have a **Theory of the Mind**. When we talk to each other, we have a theory of what the other person knows and will understand, and we answer contextually. Theory of the mind is certainly not just the prerogative of humans; it has been observed in most animals and was recently discovered even among fishes.

To the great surprise of researchers, language models turned out to have a theory of the mind too. Based on what they think the user is and how the user thinks, they calibrate their response. This complicates matters for RAG—if you ask “How many days of leave am I allowed?” and the model has the ability to infer who you are, will the answer change? The answer is: the semantic core will remain the same for factual queries. Assuming the search engine has retrieved the right document with the leave table at your level of position and title, it will still give the same answer. Its expressions may change—its language may vary—but the semantic core will remain the same. You don't have to worry that because temperature is not zero, the answer will suddenly go from 4 days to 40 days.

8 The Attention Mechanism

8.1 The Dictionary Lookup Analogy

The first exercise: Suppose you receive the garbled word “receiv.” What does it remind you of in a dictionary? Which words would you go looking for?

Candidates emerge: “receive,” “deceive,” “relieve,” “recipe.” Most students immediately gravitate toward “receive.” The instructor wrote attention scores: 95% to “receive,” 5% or 4% to “deceive,” and 1% to “recipe.”

But what made you do that? You don’t judge solely by lexical string distance. In your childhood, how many of you have misspelled “receive”? All the time. Still do it. The language models have studied the world literature and seen all the misspellings. How often would they have seen “deceive” misspelled as “receiv” versus “receive” misspelled that way? A good machine, when it gives weightage, is not doing string distance between words—it is getting a **prior belief** into the system.

The final contextual meaning is a weighted aggregation: you carry away the meaning of “receive” with a touch of “relief” and the slightest hint of “recipe.” You can never be too sure.

Example 2: “accs.” The candidates are “accent,” “across,” and “actress.” Here you have a real problem—“across” and “accent” are both plausible typos. Attention must be spread more evenly, perhaps 40%, 35%, 25%.

Example 3: “E H” (a transposition). Candidates are “the” and “ten.” Now something interesting happens: the string distance (a simple transposition of one letter) is identical for both. Traditional string distance can’t help you. But contextual statistics from vast human literature tell you that “the” is overwhelmingly more likely—roughly 85%. The machine understands that “teh” is semantically closer to “the.” This is the distinction between pure lexical distance and a semantic scoring based on experience.

Example 4: “ruinnng.” Candidates are “running,” “done,” and “ruin.” Weights: 85%, 10%, 5%. Once again, the statistics of human error inform the machine’s attention. You would say the meaning of this garbled word is “running” with a touch of “done” and spiced up with “ruin” a tiny bit. You can never be sure that it wasn’t “ruin.”

8.1.1 *The Spectrum of Meaning: Where Does It Live?*

There is a critical spectrum for understanding where a word’s meaning resides:

1. **Meaning fully in the word.** If someone says “plier,” immediately a mental image of the tool comes up. A lot of the meaning is right there in the word itself.
2. **Meaning partly in the word, partly in the context.** Polysemous words like “bank” are classic examples. “Bank” has many meanings:
 - “You banked on me to bring biryani for lunch.”
 - “You deposit your salary in the bank.”
 - “The airplane banks to land, to line up with the runway.”

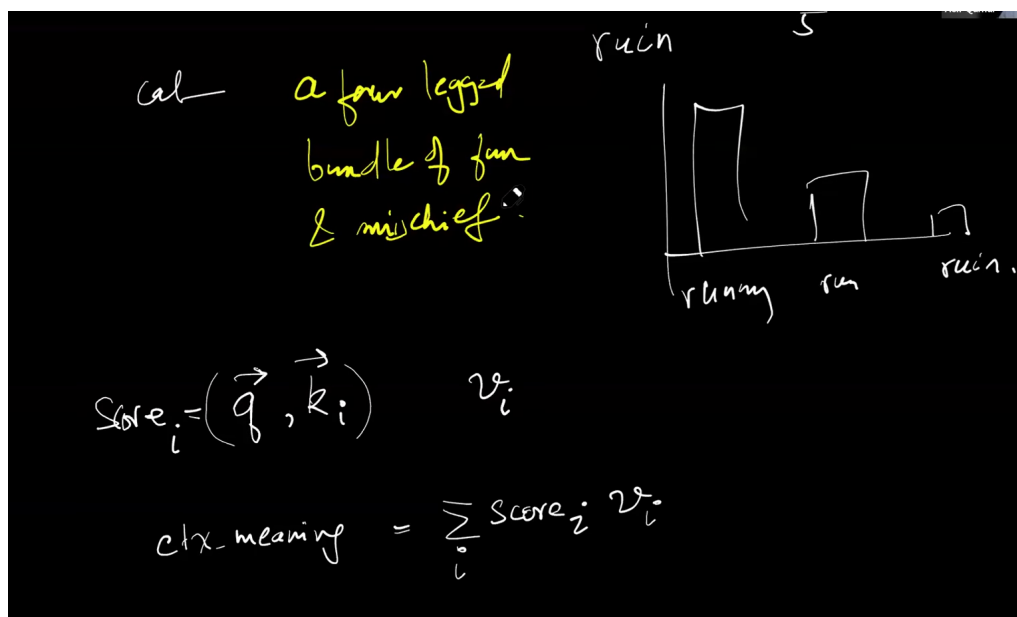


Figure 10: Finding contextual meaning.

- “Rivers have banks.”

1. **Meaning not in the word at all.** “She went and deposited her paycheck at Wells Fargo.” Is “Wells Fargo” in the dictionary? No. It’s something Californians know about, but the meaning of Wells Fargo is not in the word Wells Fargo. Yet it is amply clear what it is—a financial bank. The meaning is entirely in the context.

To different degrees, meaning is partly in the word and partly in the context—especially when you misspell.

8.2 The Attention Heat Map

Now consider the sentence: “She swam across the river to the other bank.”

If you make a square grid of words (she, swam, across, the, river, to, the, other, bank), each word goes looking for its meaning. The first act of looking should be in the mirror—because some meaning may be right there in the word itself. “Swam” might make its meaning pretty clear. “River” has quite a bit of meaning in the word itself—it might ascribe maximum attention to itself, and then some attention to other words to be sure it’s still a flowing water body.

The result is a **heat map of attention scores**. You will get a **pronounced diagonal** (each word pays strong attention to itself) but the word “bank” will be giving a lot of attention to “river” and “swam”—those are relevant words that disambiguate its meaning. “She” and “the” may not be so relevant.

To be able to create a high-fidelity, meaningful, correct heat map is the core of understanding text. If each word is completely unambiguous in what it means in context, the machine has at that moment understood the sentence.

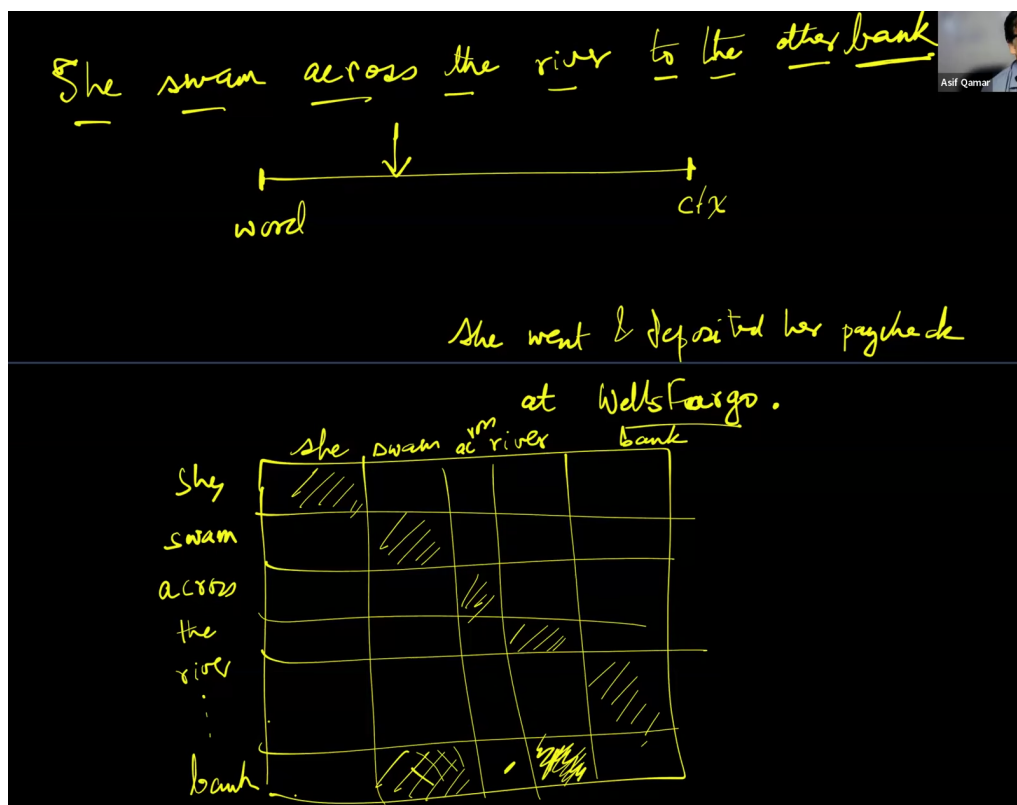


Figure 11: The Attention Heatmap

A student asked: “This happens during training, right?” Yes—training is the process by which the machine learns to create these attention score heat maps correctly. In the beginning, if a machine doesn’t know Tibetan, the word “bank” in Tibetan would have no reason to look at “river” any more than it can look at “the”—because it doesn’t know the language. But as the machine learns from more and more text, the only way it can get the next-word prediction right is if it understands how to fill up this heat map correctly.

8.3 Machines Speak Machineese: Queries, Keys, and Values

So now the question comes: machines speak machines. What is the notion of score based on proximity? If all points sit on a globe, the simplest proximity measure is the angle between them—the smaller the angle, the more proximal. A measure of that is cosine. But when the vectors are not equal length, it is the **dot product**—how aligned the two vectors are. The score function of a query with respect to a key word is basically the dot product between the query and the key.

But dot products are not normalized—they can be positive or negative. Attention scores should be positive (you can’t give negative attention to anything—like in the hiking example, where you give different amounts of attention to things in the forest, but those scores are positive degrees of emphasis). So how do you take a bunch of dot products, positivize them, and make them add up to one? You **softmax** it—exponentiate and

normalize. And then you remember something about temperature: it's a good idea to bring in temperature to modulate it. Otherwise, if a quantity is big, the exponential will just dominate it. The temperature comes in as the square root of the dimensionality for a mathematical reason. If you're doing it in 64-dimensional space (the one they initially used), $\sqrt{64} = 8$, and you just scale it down to mute things so that one pillar wouldn't just blow up.

But where do the Query, Key, and Value vectors come from? When a word is searching for its meaning, it is a Query vector. When viewed from a different lens—when other words want to figure out their meaning—it acts like the Key word in a dictionary. And yet from another lens, its current understood meaning is the Value. So we need three operators to extract three different roles from each word:

- A weight matrix W^Q to produce a Query out of a word embedding.
- A weight matrix W^K to extract a Key.
- A weight matrix W^V to extract the Value.

These are the simplest operators: linear operators, also called matrices. Matrices are operators that transform one vector into another. Think of them as operators that look at that thing from the necessary lens.

8.4 The Core Equation of Attention

$$\text{ctx_meaning (rank)} = \sum_i s_i v_i$$

$$\text{Score}_i = \text{softmax} \left(\frac{\langle q_{\text{query}}, \text{key}_i \rangle}{\sqrt{d}} \right)$$

$$\text{attention} = \sum_i \text{score}_i v_i$$

$$\text{attention} = \sum_i \text{softmax} \left(\frac{\langle q_{\text{query}}, \text{key}_i \rangle}{\sqrt{d}} \right) v_i$$

Figure 12: Deriving the Attention Equation.

The contextualized meaning is called **Attention**. The full equation is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V \quad (5)$$

Breaking it down: a query representing a word looking for its meaning has a matrix multiplication with key (the dot product). The scaling is $\sqrt{d_k}$. Then you softmax that. Then you do a matrix multiplication with V. The paper writes this same equation in matrix form.

8.5 Self-Attention and “Attention is All You Need”

The paper *Attention is All You Need* [Vaswani et al., 2017] has more than a quarter million citations.

The paper introduced the **Transformer**—a new architecture based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. In those days, attention was like pixie dust: any architecture you threw some into, it got better. The breakthrough was showing you could remove everything else and use only attention.

Recommended resources: Jay Alamar’s *Illustrated Transformer* blog (now an O’Reilly book), Harvard’s annotated transformer, and Three Blue One Brown’s videos on attention. **Multi-head attention** and deeper nuances were deferred to a separate session.

8.5.1 Self-Attention: The Small Dictionary

When you have a sentence like “She swam across the river to the other bank,” the word “bank” needs to figure out its meaning by looking at only these words. The only dictionary it needs is a small dictionary of key-values made up of just the words in the sentence. You don’t need the whole Oxford Learner’s Dictionary or any linguistic dictionary, because the meaning has to be inferred from this text. That is the **self** in self-attention: a word searches for its meaning from a small dictionary made out of only the words in the text. The text is the context.

A student raised an important point: that self-attention assumes all these words are correct. The instructor demonstrated by writing a made-up sentence: “**The vulat gravited downly but oer groed.**” Every word is searching for its meaning and scratching its head. He referenced Lewis Carroll’s *Jabberwocky*—a poem made entirely of invented words that still feels like it makes sense when read aloud. He deliberately did not use Jabberwocky as the class example because LLMs would immediately figure it out—so much has been written about it that the model already has the context. So he created a nonsense sentence of his own instead.

The limitation of self-attention: the context should be revealing enough. It mostly works—until you come across Jabberwocky or its variant.

9 Capstone Projects and The Embedding Histograms Lab

The session concluded by assigning 18 Capstone Projects and conducting an interactive lab on embedding spaces.

9.1 The Baseline RAG Capstone

Students are required to form teams and draft a 3-month project plan. The first milestone is building a raw, baseline RAG. The instructor advised starting simple: load a corpus (like StackOverflow dumps) into a traditional search engine (like ElasticSearch or OpenSearch), execute standard keyword search, and stitch it into a RAG by taking the search results and feeding them to the LLM. Students haven't yet learned chunking strategies, query transformation, guardrails, vector databases, indexing, or derivative artifacts—all deep topics to come.

9.2 The Embeddings Evaluation Lab

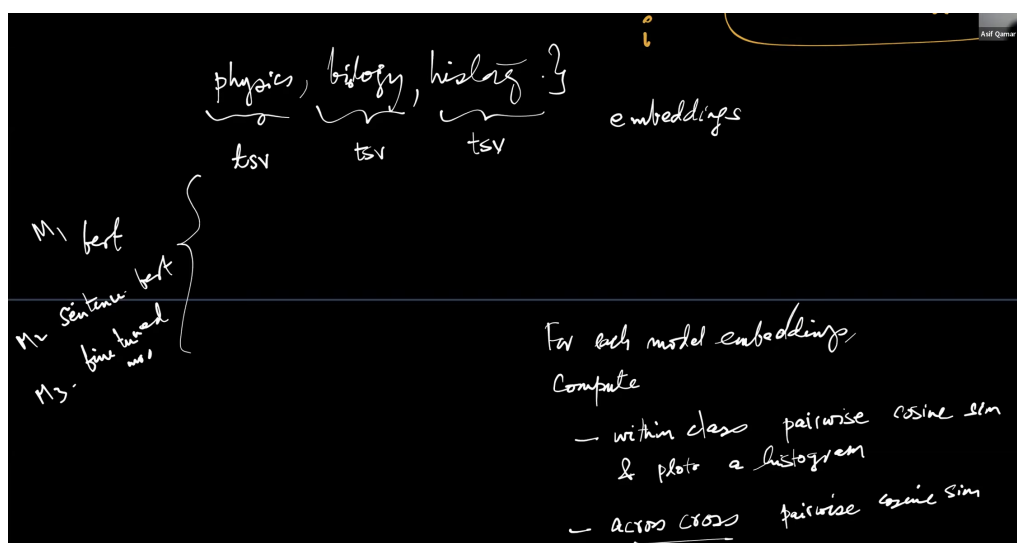


Figure 13: Instructions for the Embeddings Evaluation Lab

Dataset: The dataset consisted of TSV files containing embeddings for sentences across three subjects: **Physics, Biology, and History**.

Models: Three embedding models were evaluated—three models “speaking different dialects of machines”:

1. A BERT-like model (the instructor noted anisotropy is strongest here).
2. A generic text embedder downloaded from Hugging Face, which the instructor referred to as “Sentence BERT” (less anisotropy than model 1).

3. A model that was specifically fine-tuned to work well on this dataset (least anisotropy, strongest separation).

The Exercise: Students wrote scripts to compute the pairwise cosine similarity for vectors **Within Class** (e.g., Physics vs. Physics) and **Across Class** (e.g., Physics vs. Biology). The goal was to plot 12 histograms (4 per model) to visualize how different models separate semantic concepts.

The Finding: Anisotropy vs. Repulsion When viewing generic embeddings in the TensorFlow Embedding Projector (projector.tensorflow.org), the dots appear spread out. However, this is an optical illusion caused by projecting high dimensions down to 3D.

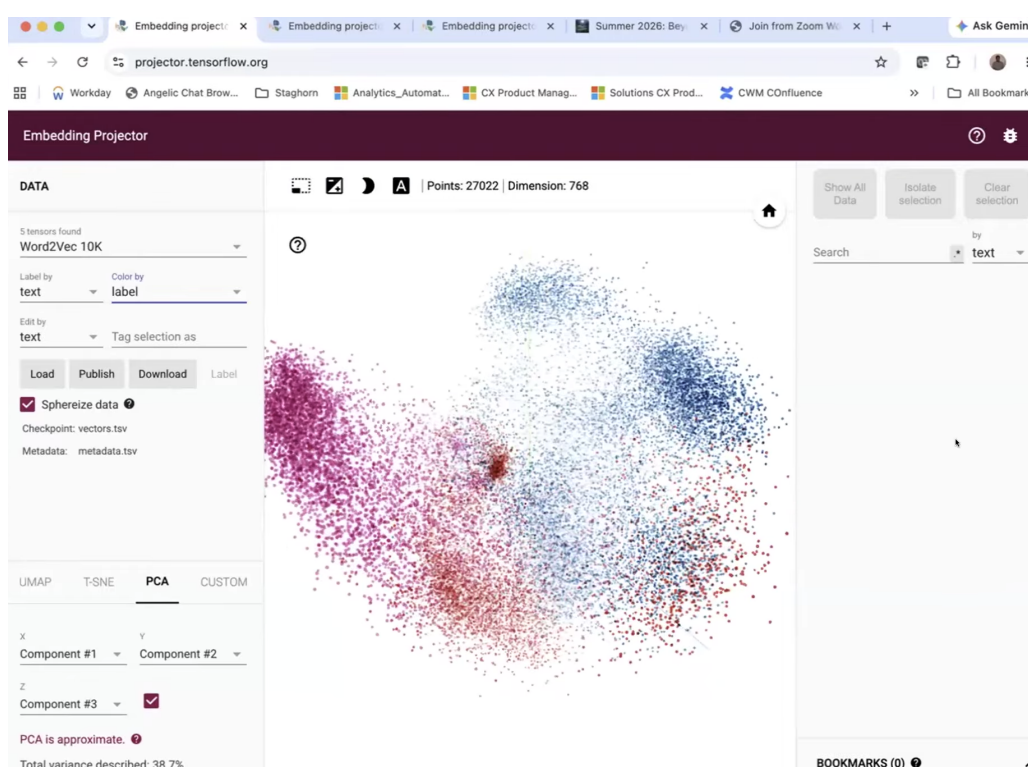


Figure 14: Anisotropic Embeddings

The histogram math proves that generic language models suffer from **Anisotropy**—they cluster all points tightly together in a narrow cone. Because all sentences are written in English, the models’ NLL loss functions create a massive attractive force centered around grammatical “Englishness” (common connective tissue and syntax), blurring the boundaries between Biology and History.

The **Fine-Tuned Model**, however, fundamentally alters the geometry of the vector space. Fine-tuning introduces a **repulsive force**. Within-class there is very strong cohesion, and across-class there is very distinct separation. The classes spread out to use up the space, positioning themselves as far apart as possible—they almost form like an equilateral triangle in space.

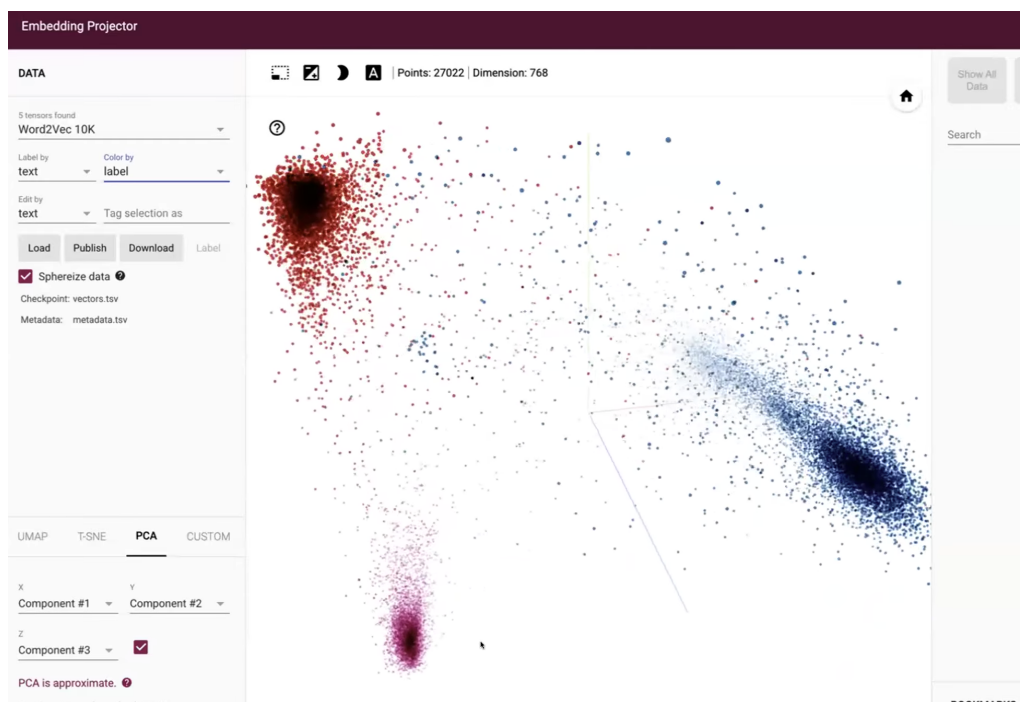


Figure 15: Embeddings from Fine-tuned model

The Ultimate Engineering Lesson: If you use a text embedder just downloaded or because your cloud provider gave you, you leave a vast opportunity unrealized on the table. Most good RAG systems start with hand-built, hand fine-tuned embedding models for specific domains and specific tasks.

10 Key Takeaways

- **The Agentic Loop:** AI intelligence is defined by a continuous cycle of sensing, reasoning, and acting within an environment, moving beyond passive chatbots.
- **RAG Architecture:** A robust pipeline decouples Search and Retrieval from the Reader (LLM) and the final Generation phases.
- **Vector Semantics:** Meaning is encoded as vectors in high-dimensional latent space, where proximity metrics like Cosine Similarity and Dot Product dictate semantic closeness.
- **The Curse of Dimensionality & Anisotropy:** In high dimensions, generic embedding models often cluster vectors too tightly due to linguistic gravity (Anisotropy), making separation difficult.
- **The Attention Mechanism:** Contextual meaning is assigned dynamically by computing the relationship between Query, Key, and Value vectors derived from the text.

- **Softmax & Temperature:** Softmax normalizes logits into probabilities. Temperature controls lexical variance, but hardware realities prevent perfect determinism even at $T = 0$.
- **Theory of the Mind:** LLMs dynamically adjust their lexical tone based on inferred user intent while preserving factual semantic consistency.
- **Fine-Tuning is Essential:** Relying on generic embedding models leaves performance unrealized. Fine-tuning introduces repulsive forces that cleanly separate semantic classes for production RAG systems.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Note: This document is a summary of Week 2 of the Beyond RAG: Advanced Agentic Knowledge Architectures course. For complete details, refer to the course portal, lab notebooks, and session recordings.
--